Multi-Source Candidate Data Transformer - Architecture & Design

This document outlines the technical design for the Eightfold Multi-Source Candidate Data Transformer, meeting the requirements for Step 1 of the assignment.

1. Pipeline Breakdown

The transformer will process candidate data through a unidirectional, 6-stage pipeline:

1. **Detect & Ingest**: The CLI accepts input files and URLs. The system detects the source type (e.g., Recruiter CSV, GitHub URL) and routes it to the appropriate extractor.
2. **Extract**: Source-specific adapters parse raw data (CSV rows, API JSON responses) into an unnormalized, flat Intermediate Representation (IR).
3. **Normalize**: Field-level sanitizers clean the IR data:
- *Phones*: Parsed and formatted to E.164 using a library like `phonenumbers`.
- *Locations*: Mapped to ISO-3166 alpha-2 country codes.
- *Dates*: Converted to `YYYY-MM`.
- *Skills*: Lowercased and mapped against a predefined canonical dictionary (e.g., "js" -> "javascript").
4. **Merge & Resolve**: Records from different sources belonging to the same candidate are combined.
- *Identity Matching*: Candidates are merged primarily based on exact `email` matches.
- *Conflict Resolution*: If sources provide conflicting data, a priority-based merge policy determines the winner.
5. **Project-to-Output**: A configuration engine takes the merged canonical profile and applies the runtime custom-output config to reshape, rename, and filter the data.
6. **Validate**: The final output is validated against the required JSON schema before being written to stdout/file.

2. Canonical Output Schema & Normalized Formats

The internal canonical profile will rigorously adhere to the schema provided in the assignment, acting as the "source of truth":

- `candidate_id` (string): Unique UUID generated for the merged profile.
- `full_name` (string)
- `emails` (string[]): Deduplicated array of valid emails.
- `phones` (string[]): Formatted to **E.164** (+14155552671).
- `location` (object): { `city`, `region`, `country`: **ISO-3166 alpha-2** }.
- `links` (object): Typed URLs for linkedin, github, portfolio.
- `skills` (array of objects): Standardized to **canonical skill names**.
- `experience`, `education` (array of objects): Dates strictly in **YYYY-MM**.
- `provenance` (array of objects): Tracks `{ field, source, method }`.
- `overall_confidence` (number): 0.0 to 1.0 score.

3. Merge / Conflict-Resolution Policy & Confidence

Match Keys:
- Primary: Exact match on `email`.
- Secondary (if time permits): Exact match on `github profile URL` or `linkedin profile URL`.

Conflict Resolution (Picking a Winner):
We will use a **Source Priority Weighting** system.
- E.g., for `skills`, GitHub API (weight: 0.9) overrides a Recruiter CSV (weight: 0.5).
- For `phone_number`, Recruiter CSV (weight: 0.9) overrides a scraped GitHub bio (weight: 0.3).
- If an array field (like `skills` or `emails`) can accept multiple values, we take the union and deduplicate.

Assigning Confidence:
- Each source adapter assigns a base confidence to its extracted fields (e.g., Structured CSV = 0.9, Unstructured Text Extraction = 0.6).
- If multiple independent sources agree on a value (e.g., both CSV and GitHub report the same email), the confidence score for that field is boosted.
- `overall_confidence` is the weighted average of the individual field confidences.

4. Runtime Custom-Output Config Handling (Projection)

The projection layer operates completely independently of the core pipeline engine.
- It evaluates the runtime JSON config.
- Iterates over the `fields` array.
- Uses dot-notation paths (e.g., `emails[0]`) via a tool like `jsonpath` to query the internal canonical profile.
- Applies any requested runtime `normalize` rules.
- Strips out `confidence` and `provenance` if `include_confidence: false`.
- Enforces the `on_missing` policy: if a field is null, it will either omit the key, leave it null, or throw an exception based on the config.

5. Edge Cases & Handling

1. **Candidate Identity Clash (Same Name, Different Person)**
- *Handling*: We never merge solely on `full_name`. Merging requires a hard unique identifier like an `email` or `profile URL`.
2. **Missing or Garbage Sources**
- *Handling*: Extractors are wrapped in safe try/catch blocks. If a GitHub API call fails (404) or a CSV is malformed, the pipeline logs a warning and proceeds with the available sources (degrades gracefully). Unknown values are assigned `null`.
3. **API Rate Limiting (GitHub)**
- *Handling*: For a production system scaling to thousands, we'd need queues, retries, and API key rotation.
- *Descoped*: Under time pressure for this assignment, we will implement simple caching but assume rate limits won't be hit for the small sample inputs.
4. **Missing Required Fields During Projection**
- *Handling*: If a runtime config marks a field as `"required": true` but the canonical profile lacks it, the pipeline will throw a Validation Error rather than failing silently, respecting the "wrong-but-confident is worse than honestly-empty" philosophy.